

二维基础

向量点积: $\vec{a} \cdot \vec{b} = a_x b_x + a_y b_y$, 几何意义: $\vec{a} \cdot \vec{b} = ||\vec{a}|| ||\vec{b}|| \cos \theta$

模长: $||\vec{a}|| = \sqrt{\vec{a} \cdot \vec{a}}$, 夹角: $\cos \theta = \frac{\vec{a} \cdot \vec{b}}{||\vec{a}|| ||\vec{b}||}$, 投影: $||\vec{a}|| \cos \theta = \frac{\vec{a} \cdot \vec{b}}{||\vec{b}||}$, 垂直: $\vec{a} \cdot \vec{b} = 0$

向量叉积: $\vec{a} \times \vec{b} = (a_x b_y - a_y b_x) \hat{n}$ ($\hat{n}$ 表示垂直于 ab 平面的法向量), 几何意义:
 $\vec{a} \times \vec{b} = ||\vec{a}|| ||\vec{b}|| \sin \theta \hat{n}$

平行四边形面积: $||\vec{a}|| ||\vec{b}|| \sin \theta = ||\vec{a} \times \vec{b}||$, 平行: $\vec{a} \times \vec{b} = 0$

```
1  #ifndef usedouble
2  using T = long double;
3  const T eps = 1e-8;
4  const T pi = 3.141592653589793238462643383279502884L;
5  #else
6  using T = long long;
7  const T eps = 0;
8  #endif
9  const T inf = numeric_limits<T>::max();
10
11 struct Point { // 点和向量
12     T x, y;
13
14     Point operator+(const Point &a) const {return {x+a.x,y+a.y};}
15     Point operator-(const Point &a) const {return {x-a.x,y-a.y};}
16     Point operator-() const {return {-x,-y};}
17     Point operator*(const T k) const {return {k*x,k*y};}
18     Point operator/(const T k) const {return {x/k,y/k};}
19 #ifndef usedouble
20     bool operator==(const Point &a) const {return (abs(x-a.x)<=eps && abs(y-
a.y)<=eps);}
21     bool operator<(const Point &a) const {return (abs(x-a.x)<=eps)?(y<a.y-
eps):(x<a.x-eps);}
22 #else
23     bool operator==(const Point &a) const {return (x==a.x && y==a.y);}
24     bool operator<(const Point &a) const {return (x==a.x)?(y<a.y):(x<a.x);}
25 #endif
26     bool operator>(const Point &a) const {return !(*this<a || *this==a);}
27
28     // 点积, 大于0: 锐角; 小于0: 钝角; 等于0: 垂直
29     T operator*(const Point &a) const {return x*a.x+y*a.y;}
30     // 叉积, 四边形有向面积; 大于0: a在this逆时针; 小于0: a在this顺时针
```

```

31     T operator^(const Point &a) const {return x*a.y-y*a.x;}
32     // 1: a在这左侧; -1: a在这右侧: 0: 共线
33     int toleft(const Point &a) const {const auto t=(*this)^a; return (t>eps)-(
(t<-eps));}
34     T len2() const {return (*this)*(*this);}
35     T dis2(const Point &a) const {return (a-(*this)).len2();}
36     // 0:原点 1:x轴正 2:第一象限 3:y轴正 4:第二象限 5:x轴负 6:第三象限 7:y轴负 8:第四
象限
37     int quad() const {
38         if(abs(x) <= eps && abs(y) <= eps) return 0;
39         if(abs(y) <= eps) return (x > eps) ? 1 : 5;
40         if(abs(x) <= eps) return (y > eps) ? 3 : 7;
41         return (y > eps) ? (x > eps ? 2 : 4) : (x > eps ? 8 : 6);
42     }
43 #ifdef usedouble
44     T len() const {return sqrtl(len2());}
45     T dis(const Point &a) const {return sqrtl(dis2(a));}
46     // this与a夹角, 范围[0,pi]
47     T ang(const Point &a) const {return acosl(max(-1.01,min(1.01,
(((*this)*a)/(len()*a.len()))));}
48     // 逆时针旋转 rad
49     Point rot(const T rad) const {return {x*cos(rad)-
y*sin(rad),x*sin(rad)+y*cos(rad)};}
50     Point rot(const T cosr,const T sinr) const {return {x*cosr-
y*sinr,x*sinr+y*cosr};}
51 #endif
52 };
53
54 struct Argcmp { // 极角排序
55     bool operator()(const Point &a,const Point &b) const {
56         const int qa = a.quad(), qb = b.quad();
57         if(qa != qb) return qa < qb;
58         const auto t = a ^ b;
59         if(abs(t) <= eps) return a.len2() < b.len2(); // 共线时按长度升序
60         return t > eps;
61     }
62 };
63
64 struct Line { // 直线
65     Point p, v; // 直线上一点 p, 方向向量 v
66
67     bool operator==(const Line &a) const {return v.toleft(a.v)==0 &&
v.toleft(p-a.p)==0;}
68     int toleft(const Point &a) const {return v.toleft(a-p);}
69     // 极角序优先; 平行时, 靠左的直线排在前面
70     bool operator<(const Line &a) const {
71         if(abs(v^a.v)<=eps && v*a.v>=-eps) return toleft(a.p)==-1;

```

```

72         return Argcmp()(v, a.v);
73     }
74
75 #ifndef usedouble
76     // 求两直线的交点, 需要确保两直线不平行  $v \wedge a.v \neq 0$ 
77     Point inter(const Line &a) const {return p+v*((a.v^(p-a.p))/(v^a.v));}
78     T dis(const Point &a) const {return abs(v^(a-p))/v.len();}
79     // 点 a 在直线上的投影点
80     Point proj(const Point &a) const {return p+v*((v*(a-p))/(v*v));}
81 #endif
82 };
83
84 struct Segment { // 线段
85     Point a, b;
86
87     bool operator<(const Segment &s) const {return make_pair(a,b)
<make_pair(s.a,s.b);}
88
89     // -1: 在端点上; 0: 不在线段上; 1: 严格在线段内部
90     int is_on(const Point &p) const {
91         if(p == a || p == b) return -1;
92         return ((p-a).toleft(p-b)==0) && ((p-a)*(p-b)<-eps);
93     }
94     // -1: 直线过线段端点; 0: 不相交; 1: 严格相交
95     int is_inter(const Line &l) const {
96         if (l.toleft(a) == 0 || l.toleft(b) == 0) return -1;
97         return l.toleft(a) != l.toleft(b);
98     }
99     // -1: 端点相交/包含; 0: 不相交; 1: 严格相交构成X型
100    int is_inter(const Segment &s) const {
101        if (is_on(s.a) || is_on(s.b) || s.is_on(a) || s.is_on(b)) return -1;
102        const Line l{a,b-a},ls{s.a,s.b-s.a};
103        return (l.toleft(s.a)*l.toleft(s.b)==-1) &&
(l.toleft(a)*ls.toleft(b)==-1);
104    }
105 #ifndef usedouble
106     T dis(const Point &p) const {
107         if((p-a)*(b-a)<-eps || (p-b)*(a-b)<-eps) return min(p.dis(a),
p.dis(b));
108         const Line l{a, b - a};
109         return l.dis(p);
110     }
111     T dis(const Segment &s) const {
112         if(is_inter(s)) return 0;
113         return min({dis(s.a), dis(s.b), s.dis(a), s.dis(b)});
114     }
115 #endif

```

```

116 };
117
118 struct Polygon { // 一般多边形
119     vector<Point> p; // 一般逆时针存储
120     size_t nxt(const size_t i) const {return i==p.size()-1?0:i+1;}
121     size_t pre(const size_t i) const {return i==0?p.size()-1:i-1;}
122
123     // first: 表示点是否在多边形边上
124     // second(回转数): 0 表示点在多边形外, 否则点在多边形内
125     pair<bool,int> winding(const Point &a) const {
126         int cnt = 0;
127         for(size_t i = 0, sz = p.size(); i < sz; i++) {
128             const Point u = p[i], v = p[nxt(i)];
129             if(abs((a-u)^(a-v))<=eps && (a-u)*(a-v)<=eps) return {true,0};
130             if(abs(u.y-v.y)<=eps) continue;
131             const Line uv = {u, v - u};
132             if(u.y<v.y-eps && uv.toleft(a)<=0) continue;
133             if(u.y>v.y+eps && uv.toleft(a)>=0) continue;
134             if(u.y<a.y-eps && v.y>=a.y-eps) cnt ++;
135             if(u.y>=a.y-eps && v.y<a.y-eps) cnt --;
136         }
137         return {false,cnt};
138     }
139     T area2() const { // 面积两倍; 面积为正表示点集逆时针存储
140         T sum = 0;
141         for(size_t i = 0, sz = p.size(); i < sz; i++)
142             sum += (p[i] ^ p[nxt(i)]);
143         return sum;
144     }
145 #ifdef usedouble
146     T circ() const { // 周长
147         T sum = 0;
148         for(size_t i = 0, sz = p.size(); i < sz; i++)
149             sum += p[i].dis(p[nxt(i)]);
150         return sum;
151     }
152 #endif
153 };

```

